

پرسش ۱ (۲ نمره) – توابع بازگشتی به دلیل فراخوانی‌های تودرتوی زیاد، بالقوه فضای زیادی را از پشته (stack) اشغال می‌کنند. چه روشی می‌تواند تا حدی این مشکل را کاهش دهد؟

با تعریف پارامترهای بزرگ به شکل ارجاع ثابت (const reference) می‌توان حافظه لازم برای هر فراخوانی را کاهش داد.

پرسش ۲ (۲ نمره) – چرا برنامه‌نویسی شیء‌گرا تغییر دادن کد را ساده‌تر می‌کند؟

با مخفی کردن پیاده‌سازی کلاس‌ها وابستگی بخش‌های دیگر به داخل یک کلاس را کم می‌کند و در نتیجه تغییر در آن را ساده می‌کند.

پرسش ۳ (۴ نمره) – چهار خطای زمان کامپایل را در کد زیر مشخص نمایید و درباره دلیل آن مختصراً توضیح دهید.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;

5  class Book {
6  private:
7      string title;
8      int year;
9  public:
10     Book(const string& t, int y) {
11         title = t;
12         year = y;
13     }
14     bool published_in_the_same_year(Book book);
15 };

17 bool published_in_the_same_year(Book book) { // Missing year::
18     return year == book.year;
19 }

20 int main() {
21     Book b("Rebecca", 1938);
22     vector<Book> books(100); // No default constructor defined for Book
23     books.push_back(b);
24     books.push_back(Book("Brighton Rock", 1938));
25     if (b.published_in_the_same_year(&books[1])) // Wrong to pass address
26         cout << books[1].title(); // No method named title()
27 }
```

پرسش ۴ – در این پرسش با رشته‌هایی سروکار داریم که فقط از حروف کوچک انگلیسی تشکیل شده‌اند. یک رشته را مطلوب می‌نامیم اگر در آن هیچ دو حرف بی‌صدایی کنار هم قرار نگرفته باشند. مثلاً "hello" مطلوب نیست اما "helo" مطلوب است. رشته‌های به طول صفر یا یک مطلوب هستند. فرض کنید حروف صدا دار انگلیسی aeiou هستند. توابع زیر را به صورت بازگشتی و بدون استفاده از حلقه بنویسید.

(الف – ۲ نمره) تابعی بازگشتی با امضای `bool matloob(string s)` بنویسید که تعیین کند آیا رشته `s` مطلوب است یا نه.

```

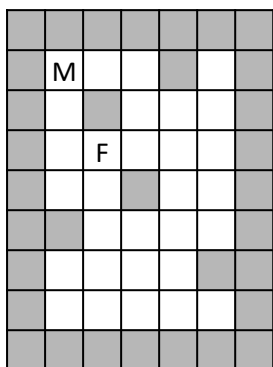
bool is_vowel(char c) {
    return string("aeiouy").find(c) != string::npos;
}
```

```
bool matloob(string s) {
    if (s.length() < 2)
        return true;
    if (!is_vowel(s[0]) && !is_vowel(s[1]))
        return false;
    return matloob(s.substr(1));
}
```

(ب - ۳ نمره) قرار است تابعی به نام matloobed بنویسیم که یک رشته را بگیرد و مطلوب شده آن را برگرداند. به این منظور اولین زوج حرف بی صدای مجاور را پیدا می کنیم و بین آنها آخرین حرف صداداری که دیده ایم را درج می کنیم. اگر قبل از آنها هیچ حرف صداداری ندیده بوده ایم حرف a را درج می کنیم. برای روشن شدن می توانید مثال های صفحه بعد را ببینید.

```
matloobed("m") == "m"           matloobed("aardvark") == "aaradavarak"
matloobed("ma") == "ma"         matloobed("strength") == "satarenegeteh"
matloobed("mn") == "man"        matloobed("instruction") == "inisitirucution"
```

```
string matloobed(string s, char last_vowel = 'a') {
    if (s.length() < 2)
        return s;
    if (is_vowel(s[0]))
        return s[0] + matloobed(s.substr(1), s[0]);
    if (is_vowel(s[1]))
        return s.substr(0, 2) + matloobed(s.substr(2), s[1]);
    return s[0] + (last_vowel + matloobed(s.substr(1), last_vowel));
}
```



فرض کنید یک محوطه به شکل یک جدول مستطیلی داریم که از یخ پوشیده شده است. در بعضی خانه های این جدول دیوار قرار دارد که نمی توان وارد آنها شد. دور محوطه (سطرها و ستون های اول و آخر) به طور کامل از دیوار پوشیده شده اند. در این محوطه یک موش زندگی می کند که به دنبال غذا می گردد. در بعضی خانه های جدول غذا قرار دارد. موش از یکی از خانه های جدول شروع به حرکت می کند و در یکی از چهار جهت بالا، پایین، راست و چپ شروع به لغزیدن روی یخ ها می کند تا جایی که یک دیوار مانع لغزیدن بیشتر او شود. موش دوباره یک جهت را انتخاب می کند و به حرکت خود ادامه می دهد. سؤال این است که با یک محوطه داده شده آیا موش می تواند به غذا برسد یا نه. دقت نمایید برای دستیابی به غذا موش باید در خانه ای که در آن غذا هست متوقف شود و صرف رد شدن از روی آن خانه فایده ای ندارد. مثلاً در جدول مقابل حرف M موقعیت موش و F موقعیت غذا را نشان می دهد. در این حالت موش می تواند با حرکت به پایین، راست و بالا به غذا برسد. اما دنباله راست، پایین و چپ صرفاً از روی غذا رد می شود.

فرض کنید کلاسی به نام Field داریم که محوطه را مدل سازی می کند. واسط این کلاس به این شکل است.

```
class Field {
public:
    Field(int rows, int cols);    // random generation of a field with the given size
    bool hasFoodAt(Position p);
    bool hasBlockAt(Position p);
    void markAsVisited(Position p);
    void markAsUnvisited(Position p);
    bool isMarkedAsVisited(Position p);
    Position slide(Position current, int direction); // direction is in the range [0,3]
private:
    // hidden implementation
};
```

تایپ Position به عنوان یک struct تعریف شده که دو فیلد row و col دارد و عملگر == برای آن تعریف شده است. متغیرهای این تایپ خانه‌های جدول را مشخص می‌کنند. متدهای markAsUnvisited و markAsVisited یک خانه را به عنوان ملاقات شده/نشده علامت می‌گذارند و در الگوریتم پس‌گرد (backtracking) کاربرد دارند. متد slide مشخص می‌کند اگر موش از خانه current در جهت direction حرکت کند در چه خانه‌ای متوقف می‌شود (در صورتی که از ابتدا در جهت داده شده مجاور دیوار باشد همان خانه current به عنوان نتیجه برمی‌گردد).

پرسش ۵ (۴ نمره) – تابعی با امضای bool can_get_food(Field& field, Position start) تعریف کنید که به شیوه پس‌گرد تعیین کند اگر در محوطه field موش از موقعیت start شروع کند آیا می‌تواند به غذا برسد یا نه. در این مرحله نیازی به پیاده‌سازی اعضای کلاس Field نیست اما عضو جدیدی هم نمی‌توانید به آن اضافه کنید.

```
bool solve(Field& field, Position start) {
    if (field.hasFoodAt(start))
        return true;
    for (int direction = 0; direction < 3; direction++) {
        Position new_pos = field.slide(start, direction);
        if ((new_pos == start) || field.isMarkedAsVisited(new_pos))
            continue;
        field.markAsVisited(new_pos);
        if (solve(field, new_pos))
            return true;
        field.markAsUnvisited(new_pos);
    }
    return false;
}
```

پرسش ۶ – فرض کنید بخواهیم اعضای کلاس Field را پیاده‌سازی کنیم. به پرسش‌های زیر پاسخ دهید.

(الف- ۲ نمره) برای نگهداری محتویات محوطه (محل دیوارها و غذاها) دو پیاده‌سازی ممکن را پیشنهاد کنید. هر یک از این پیاده‌سازی‌ها چه مزیتی بر دیگری دارند؟

در پیاده‌سازی اول یک ماتریس (برداری از بردار) تعریف می‌کنیم که مشخص کند در هر خانه از محوطه دیوار، غذا یا یخ است (با یک کدگذاری عددی برای هر یک از موارد). از این ماتریس می‌توان برای علامت‌گذاری خانه‌های ملاقات شده نیز استفاده کرد. این پیاده‌سازی ساده‌تر است و در اجرا سریع‌تر اما برای محوطه‌های بزرگ فضای حافظه زیادی را اشغال می‌کند.

در پیاده‌سازی دیگر، یک لیست از زوج‌های محل و دیوار یا غذا نگه می‌داریم و برای تشخیص این که در یک محل غذا یا دیوار است در این لیست جستجو می‌کنیم. مزایا و معایب این روش برعکس روش قبل است.

(ب- ۱ نمره) برای پیاده‌سازی متدهای مربوط به علامت‌گذاری خانه‌ها، یک برنامه‌نویس مدعی است راهی پیدا کرده که هم از نظر زمان و هم از نظر حافظه بهینه است. برای این کار یک بردار از Positionها تعریف می‌کند که فهرست خانه‌های ملاقات شده است. هنگامی که یک خانه جدید ملاقات می‌شود آن را در انتهای بردار push_back می‌کند. از طرف دیگر چون در الگوریتم پس‌گرد همیشه آخرین حرکت انجام شده برگردانده می‌شود، برای علامت‌گذاری به عنوان ملاقات نشده، آخرین عنصر را pop_back می‌کند. به این ترتیب ضمن این که اتلاف حافظه نداریم زمان اجرا نیز پایین است. مشکل این تصمیم کجاست؟

هر چند استدلال برنامه‌نویس درباره اضافه/حذف در انتهای لیست صحیح است، اما ممکن است در طول اجرای الگوریتم نیاز باشد بررسی کنیم آیا قبلاً یک خانه را ملاقات کرده‌ایم یا نه (عبارت field.isMarkedAsVisited(new_pos)). در این حالت ناچار به بررسی کل خانه‌های ملاقات شده هستیم که از نظر حافظه بهینه نیست (نگهداری اطلاعات ملاقات در یک ماتریس سریع‌تر است).